Foreldre–barn-rolle og tilstand i spillekveld-appen

Spør heller tilstandsklassen, og send svaret ned til kortet som parameter.

GameNightState.cs

```
// GameNightState.cs
public class GameNightState
{
    private readonly List<BoardGame> games = new();

    public IReadOnlyList<BoardGame> Games => games;
    public int TotalPlayTime => games.Sum(g => g.PlayTimeMinutes);

    public event Action? OnChange;

    public bool Contains(int gameId) => games.Any(g => g.Id == gameId);

    public bool Add(BoardGame game)
    {
        if (Contains(game.Id)) return false;
        games.Add(game);
        OnChange?.Invoke();
        return true;
    }

    public void Remove(int gameId)
    {
        games.RemoveAll(g => g.Id == gameId);
        OnChange?.Invoke();
    }
}
```

GameCard.razor

```
@* GameCard.razor: viser data og melder fra, avgjør ingenting *@
<div class="game-card">
    <h3>@BoardGame.Name</h3>
    <DiceRating Value="BoardGame.Rating" ReadOnly="true" />

    @if (IsSelected)
    {
        <span>✓ Valgt</span>
        <button @onclick="() => OnRemove.InvokeAsync(BoardGame)">Fjern</button>
    }
    else
    {
        <button @onclick="() => OnAdd.InvokeAsync(BoardGame)">Legg til i spillekveld</button>
    }

    <a href="/games/@BoardGame.Id">Detaljer</a>
</div>

@code {
    [Parameter, EditorRequired] public BoardGame BoardGame { get; set; } = default!;
    [Parameter] public bool IsSelected { get; set; }
    [Parameter] public EventCallback<BoardGame> OnAdd { get; set; }
    [Parameter] public EventCallback<BoardGame> OnRemove { get; set; }
}
```

Games.razor

```
@* Games.razor: forelderen eier beslutningen og bekreftelsen *@
@inject GameNightState GameNight

@foreach (var game in filteredGames)
{
    <GameCard BoardGame="game"
              @key="game.Id"
              IsSelected="GameNight.Contains(game.Id)"
              OnAdd="HandleAdd"
              OnRemove="HandleRemove" />

    @if (messages.TryGetValue(game.Id, out var msg))
    {
        <p>@msg</p>
    }
}

@code {
    private readonly Dictionary<int, string> messages = new();

    private async Task HandleAdd(BoardGame game)
    {
        messages[game.Id] = GameNight.Add(game)
            ? "Spillet ble lagt til"
            : "Spillet er allerede lagt til";

        await Task.Delay(3000);
        messages.Remove(game.Id);
    }

    private void HandleRemove(BoardGame game) => GameNight.Remove(game.Id);
}
```

Slik oppfyller dette kravene

Krav	Hvor det løses
Kortet sier fra via `EventCallback`	`OnAdd` / `OnRemove` i `GameCard`
Kortet avgjør ikke selv hva som skjer	`GameCard` kaller bare `InvokeAsync`, uten logikk
Forelderen viser bekreftelsen	`messages` i `Games.razor`
`GameCard` viser om spillet er valgt	`IsSelected`-parameteren, sendt ned fra forelderen
Samme spill kan ikke legges til to ganger	`GameNightState.Add` sjekker `Contains`
Tilstanden har ett hjem	Bare `GameNightState` vet hva som er valgt

Gjenstår for oppgave 3

To ting gjenstår: `GameNightState` må registreres som tjeneste i `Program.cs`, og nav-menyen må abonnere på `OnChange` og melde seg av i `Dispose`.

I README kan du nevne at `messages` er et eksempel på **lokal tilstand** (den gjelder bare siden og kan forsvinne), mens spillekvelden er **applikasjonstilstand**.